PM Leads - Project Context

What We're Building

An automated lead generation pipeline for a Denver property management company. The goal is to scrape rental listings, identify DIY landlords (not using a PM company), enrich their contact info, and send cold email outreach to convert them into clients.

Tech Stack

- **Apify** — scraping (Zillow ZIP search + detail scraper)
- **Supabase** — Postgres database
- **n8n** — workflow automation
- **Clay** — contact enrichment (name + phone → email)
- **Instantly.ai** — cold email sequences
- **Python** — glue scripts

Current State

Working on `zillow_scraper.py` — a Python script that:

1. Calls Apify ZIP search actor to get Zillow rental listing URLs for Denver zip codes
2. Passes URLs to Apify detail scraper to get full listing data
3. Filters for `isListedByOwner = true`
4. Writes results to `owner_leads.csv`

Steps 1 and 2 are working. Step 3 is returning 0 results — we need to debug why `isListedByOwner` isn't matching.

Current Issue

The detail scraper returns listings but filtering on `isListedByOwner == True` returns 0 results. We need to:

1. **Print the raw first listing to see actual field names and values**
2. **Check what `isListedByOwner` actually returns (boolean vs string vs different field name)**
3. **Fix the filter accordingly**

Current Script

```
import requests
import csv
import time
import os
from dotenv import load_dotenv

# Load API token from .env file
load_dotenv()
API_TOKEN = os.getenv("APIFY_API_TOKEN")

# The two Apify actors we're using
ZIP_SEARCH_ACTOR = "maxcopell~zillow-zip-search"
DETAIL_ACTOR = "maxcopell~zillow-detail-scraper"

# Denver zip codes to search
ZIP_CODES = ["80203", "80209", "80218", "80206"]

def run_actor(actor_id, input_data):
    """Start an Apify actor run and return the run ID"""
    url = f"https://api.apify.com/v2/acts/{actor_id}/runs"
    headers = {"Authorization": f"Bearer {API_TOKEN}"}
    response = requests.post(url, json=input_data, headers=headers)
    run = response.json()
    return run["data"]["id"], run["data"]["defaultDatasetId"]

def wait_for_run(run_id):
    """Poll until the actor run finishes"""
    url = f"https://api.apify.com/v2/actor-runs/{run_id}"
    headers = {"Authorization": f"Bearer {API_TOKEN}"}

    while True:
        response = requests.get(url, headers=headers)
        status = response.json()["data"]["status"]
        print(f"Status: {status}")
        if status in ["SUCCEEDED", "FAILED", "ABORTED"]:
            return status
        time.sleep(5)
```

```
def get_dataset_items(dataset_id):
    """Fetch results from a completed run"""
    url = f"https://api.apify.com/v2/datasets/{dataset_id}/items"
    headers = {"Authorization": f"Bearer {API_TOKEN}"}
    response = requests.get(url, headers=headers)
    return response.json()

def extract_owner_fields(listing):
    """Pull only the fields we care about from a listing"""
    attribution = listing.get("attributionInfo", {}) or {}
    contact = listing.get("postingContact", {}) or {}
    address = listing.get("address", {}) or {}

    return {
        "owner_name": contact.get("name"),
        "owner_phone": attribution.get("brokerPhoneNumber"),
        "is_listed_by_owner": listing.get("isListedByOwner"),
        "address": address.get("streetAddress"),
        "city": address.get("city"),
        "state": address.get("state"),
        "zip_code": address.get("zipcode"),
        "price": listing.get("price"),
        "bedrooms": listing.get("bedrooms"),
        "bathrooms": listing.get("bathrooms"),
        "sqft": listing.get("livingArea"),
        "home_type": listing.get("homeType"),
        "days_on_zillow": listing.get("daysOnZillow"),
        "listing_url": listing.get("hdpUrl"),
        "description": listing.get("description"),
    }

def main():
    print("Step 1: Running zip code search...")
    run_id, dataset_id = run_actor(ZIP_SEARCH_ACTOR, {
        "forRent": True,
        "forSaleByAgent": False,
        "forSaleByOwner": False,
        "sold": False,
        "zipCodes": ZIP_CODES,
        "priceMin": 1500,
        "daysOnZillow": ""
    })

    status = wait_for_run(run_id)
    if status != "SUCCEEDED":
        print(f"Zip search failed with status: {status}")
```

```
    return

print("Fetching URLs from zip search...")
search_results = get_dataset_items(dataset_id)

urls = []
for item in search_results:
    url = item.get("url") or item.get("hdpUrl") or item.get("detailUrl")
    if url:
        if url.startswith("/"):
            url = "https://www.zillow.com" + url
        urls.append(url)

print(f"Found {len(urls)} listings")

print("Step 2: Running detail scraper...")
run_id, dataset_id = run_actor(DETAIL_ACTOR, {
    "startUrls": [{"url": u} for u in urls[:30]]
})

status = wait_for_run(run_id)
if status != "SUCCEEDED":
    print(f"Detail scraper failed with status: {status}")
    return

print("Fetching listing details...")
listings = get_dataset_items(dataset_id)

# DEBUG - remove once filter is working
print(f"Total listings returned: {len(listings)}")
if listings:
    first = listings[0]
    print("Sample keys:", list(first.keys())[:15])
    print("isListedByOwner value:", first.get("isListedByOwner"))
    print("isListedByOwner type:", type(first.get("isListedByOwner")))

print("Step 3: Filtering owner listings...")
owner_listings = [
    extract_owner_fields(l) for l in listings
    if l.get("isListedByOwner") == True
]
print(f"Found {len(owner_listings)} owner-listed properties")

if not owner_listings:
    print("No owner listings found - check debug output above")
    return
```

```
    print("Step 4: Writing to CSV...")
    with open("owner_leads.csv", "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=owner_listings[0].keys())
        writer.writeheader()
        writer.writerows(owner_listings)

    print("Done. Results saved to owner_leads.csv")

if __name__ == "__main__":
    main()
```

Database Schema (Supabase - not yet created)

```
CREATE TYPE property_type AS ENUM (
  'sfr', 'duplex', 'triplex',
  'fourplex', 'apartment', 'condo', 'townhouse'
);

CREATE TYPE outreach_status AS ENUM (
  'not_contacted', 'emailed', 'replied',
  'call_booked', 'signed', 'dead'
);

CREATE TYPE outreach_event_type AS ENUM (
  'email_sent', 'email_opened', 'email_replied',
  'follow_up_sent', 'call_booked',
  'call_completed', 'signed', 'dead'
);

CREATE TABLE listings (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  address_1         TEXT NOT NULL,
  city              TEXT,
  zip_code          TEXT,
  property_type     property_type,
  bedroom_count     INT,
  bathroom_count    INT,
  sqft              INT,
  advertised_rent   INT,
  owner_name        TEXT,
  owner_email       TEXT,
  owner_phone       TEXT,
  mailing_state     TEXT,
  source            TEXT,
  listing_url       TEXT,
```

```
  is_diy            BOOLEAN,
  brokerage_name    TEXT,
  first_seen_date   TIMESTAMP DEFAULT now(),
  last_seen_date    TIMESTAMP DEFAULT now(),
  times_seen        INT DEFAULT 1,
  outreach_status   outreach_status DEFAULT 'not_contacted',
  outreach_date     TIMESTAMP,
  follow_up_date    TIMESTAMP,
  notes             TEXT,
  created_at        TIMESTAMP DEFAULT now()
);

CREATE TABLE outreach_events (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id      UUID REFERENCES listings(id),
  owner_email     TEXT NOT NULL,
  event_type      outreach_event_type,
  note            TEXT,
  created_at      TIMESTAMP DEFAULT now()
);
```

Key Zillow Data Findings

- `isListedByOwner` — boolean, true = DIY landlord
- `attributionInfo.brokerPhoneNumber` — owner phone when no agent
- `attributionInfo.brokerName` — null for DIY listings
- `postingContact.name` — owner name (sometimes null in JSON, present on page)
- `priceHistory` — full listing/removal history, useful for re-listing detection
- `daysOnZillow` — clean integer
- No email available from Zillow — need Clay enrichment

Channel Strategy

- **Zillow** — primary channel, best data, DIY signal built in
- **Craigslist** — tested, no contact info available from scrapers
- **Denver Rental Registry** — 37k records but no contact info, dropped
- **Clay** — enrichment layer to get email from name + phone

- **Instantly.ai** — email sequencing

Outreach Logic (to be built in n8n)

- New owner_email → send intro email
- Existing owner_email → update record, no new email
- Re-listing signal (days_on_zillow resets, new priceHistory entry) → re-engagement sequence
- Check owner_email across ALL listings before sending — one email per owner regardless of property count

Next Steps

1. Fix `isListedByOwner` filter in Python script
2. Get clean CSV with 20-30 owner leads
3. Test Clay enrichment match rate on name + phone
4. Run Supabase schema
5. Build n8n pipeline